One-Way Door

A GITHUB COPILOT SKILL · 0:00–0:30

Every merge is a door. Some only open one way.

Before you merge, One-Way Door answers the question no pipeline asks:

> **“If this goes wrong after deploy, can we actually roll back?”**

⇄ **Two-way door: roll back any time**

→ **One-way door: no way back**



THE PROBLEM

Rollback is the safety net nobody tests

Release ships
changes a table, an event or a format

›

New state is written
rows migrated, events published

›

Something breaks
the plan: roll back

›

Code goes back
the data stays

›

Old code, new state
won't start, or silently wrong

D

Databases

Column renamed or converted in place.

Old code won't start.

E

Events

A field removed or its units changed.

Old reader sees zero, silently.

C

Caches and keys

Entries written in a new format.

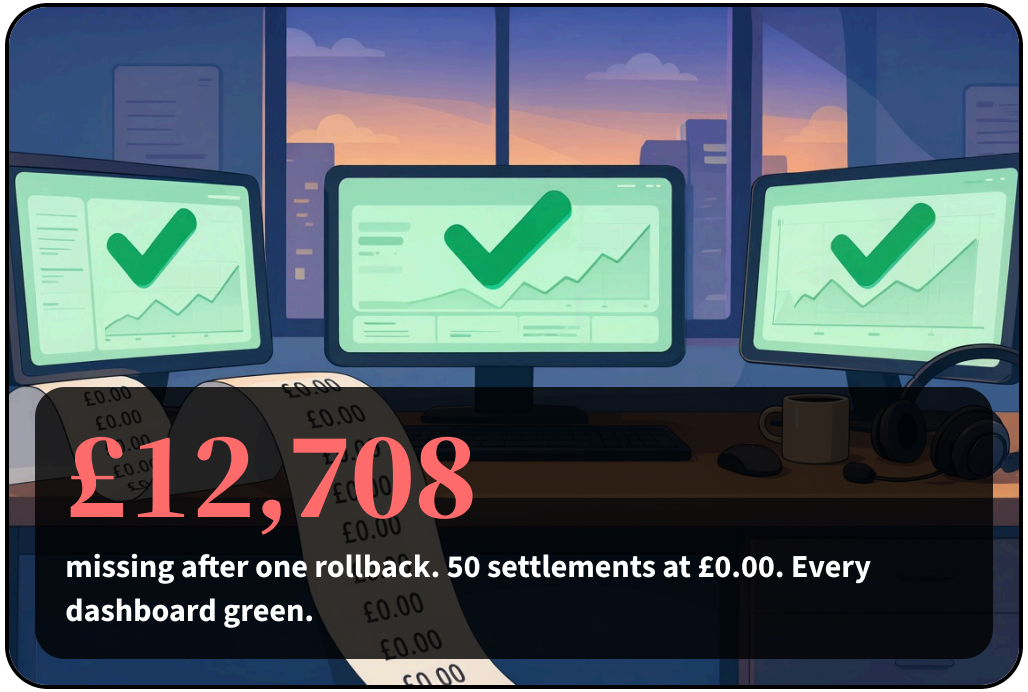
Misses, or work done twice.

X

External contracts

Webhooks sent in a new shape.

Can't be recalled.



Why it gets through: tests, CI, migration tools, code review and monitoring all look **forward**. None of them runs the old code against the new data.

THE SOLUTION

One folder that makes Copilot ask first

`.github/skills/one-way-door` loads when a PR touches a migration or an event, or someone asks “can we roll back?”.

1 Classify
Each change: two-way or one-way.

2 Time it
When the door closes, what breaks, the safe window.

3 Blast radius
Who else reads the data, even outside the PR.

4 Split it
Expand → migrate → contract → drop, generated on request.

Scripts check
offline, deterministic, JSON

+

Copilot judges
readers, old code vs new state

=

A verdict
as a PR comment

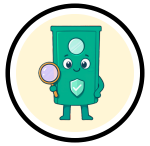
GitHub Copilot reviewed PR #482 · used one-way-door

ONE-WAY Safe rollback window: **none**

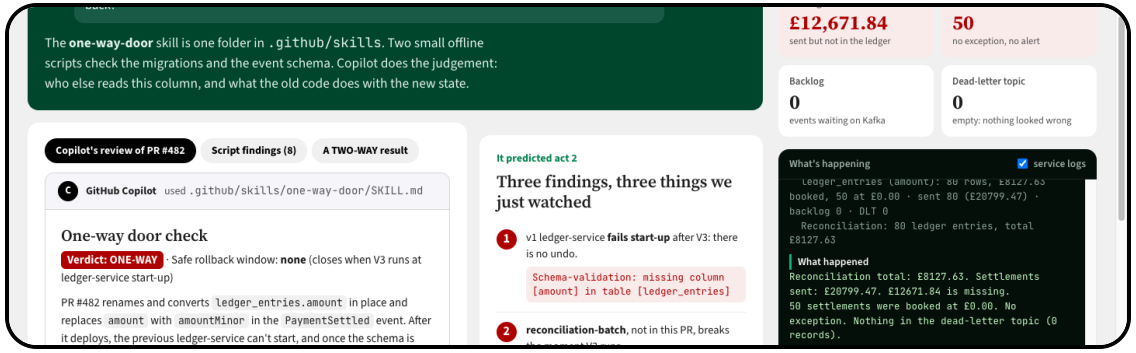
1. Column renamed and converted in place: the old ledger **won't start**.
2. Events already on Kafka **can't be rolled back**.
3. A missing field becomes **£0.00, silently**.

Blast radius: reconciliation-batch **(not in this PR)**

Suggested split: 4 PRs, each reversible.

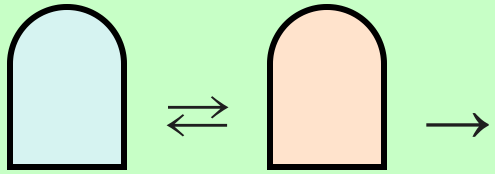


Every prediction matched a real failure in the live demo.



FOUR IDEAS, ONE VOCABULARY

How the skill thinks about a change



Two-way or one-way door

Two-way: deploy it, roll it back, the old code still works. One-way: it leaves state the old code can't handle.



When the door closes

The deploy moment after which rollback stops being safe. Here: the 5 ms it takes Flyway to run V3.



ledger · reconciliation · reports

Blast radius

Every reader of the same table, column, topic or field. The one that breaks first is often not in the PR.

1 Expand	2 Migrate	3 Contract	4 Drop
add the new	copy the data	switch readers	remove the old

The rollback-safe split

The same change in four deploys. Proven live: **4 steps PASS** the rollback test; PR #482 **FAILS** at stage 2.

APPLICABILITY

Anywhere state outlives the code

What it checks

- SQL migrations: Flyway, Liquibase, EF Core
- Event schemas: JSON Schema
- Caches, idempotency keys, external contracts
- 24 one-way patterns, each with a safe split

Where it runs

- **VS Code** agent mode, before you push
- **Copilot CLI**
- **Cloud agent** on the PR
- **Code review** with repository skills

Who it protects



- **Authors**: fix it before review
- **Reviewers**: see readers outside the diff
- **On-call**: know if rollback helps
- **Finance and data**: totals that don't drift

Adopt it in one step

```
cp -R .github/skills/one-way-door \
   your-repo/.github/skills/
```

Needs git and Node 20+. No servers, no accounts, no network calls.

Know before you merge whether you can roll back.

Next: readers in other repositories, with anything it can't see assumed to read the data.

